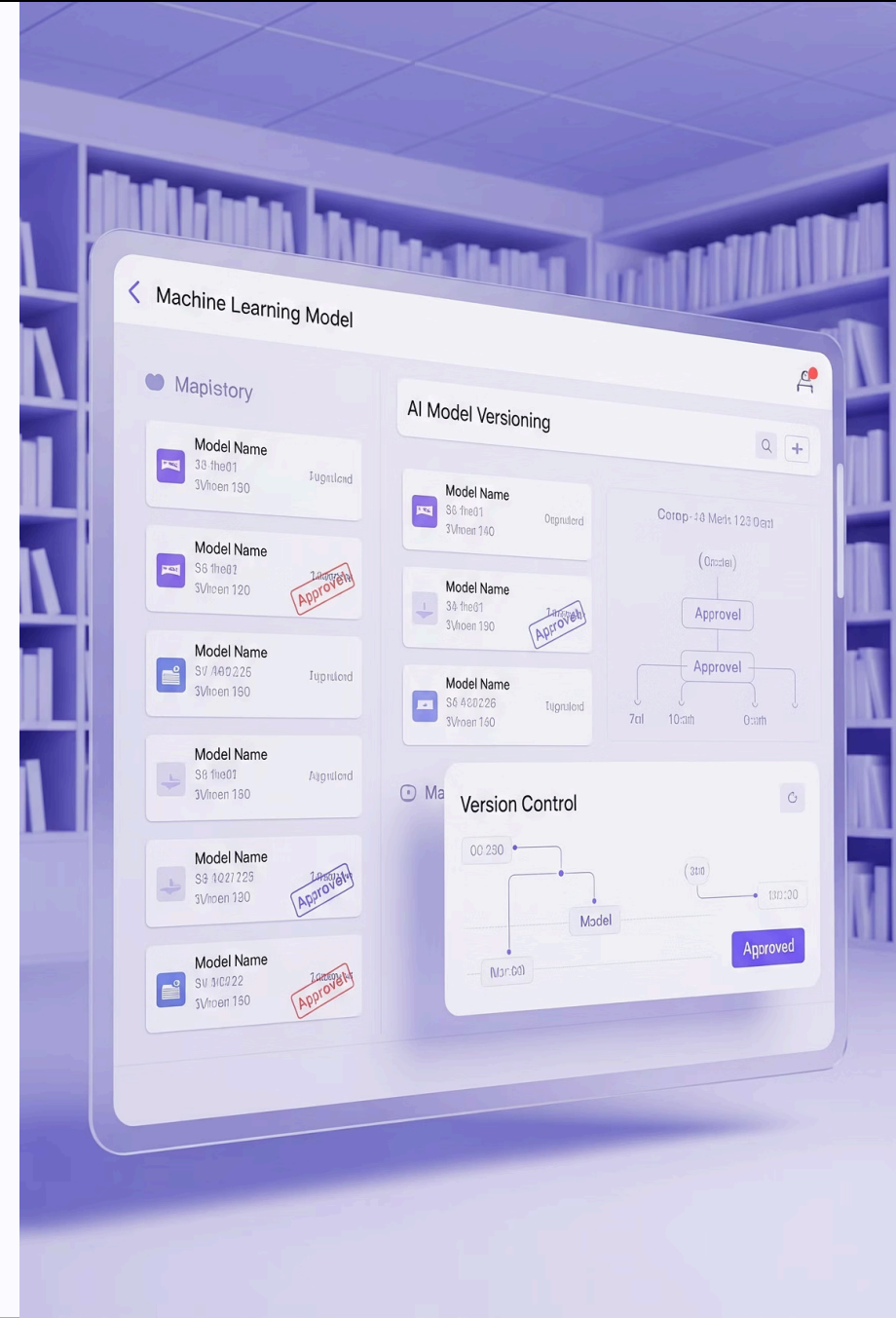


Model Registry in Enterprise MLOps

Single Source of Truth for Models, Metadata, and Promotion

A comprehensive framework for managing the full lifecycle of machine learning models in enterprise environments



Why a Model Registry?



Central Repository

Provides a single source of truth for **versions, lineage, and approvals**, eliminating confusion about which model version is in which environment.



Reproducibility

Enables complete **reproducibility** by linking models to their training data and code, essential for scientific rigor and debugging.



Standardized Promotion

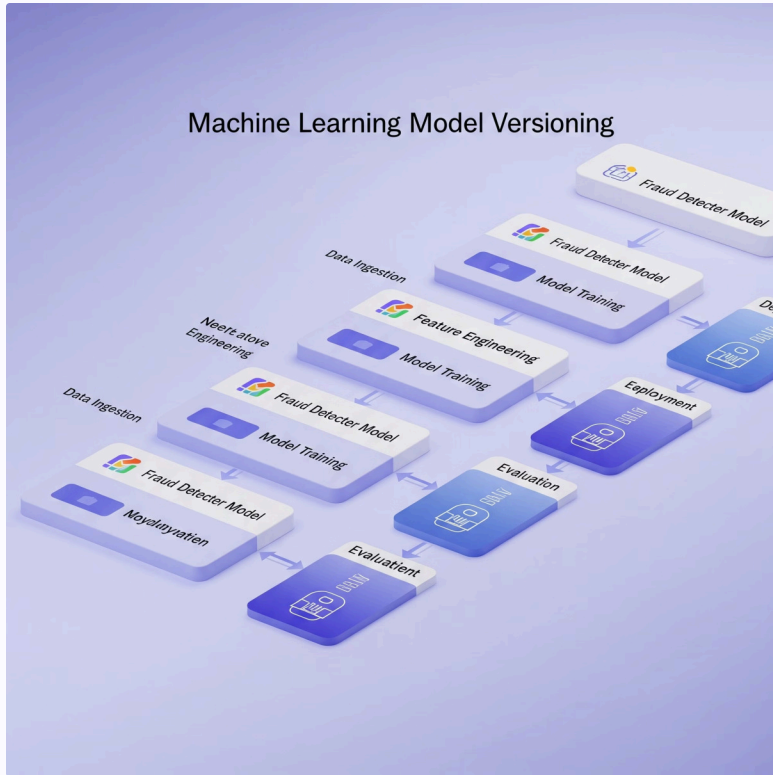
Establishes a consistent process for **promoting** models from development to staging to production environments.



Governance Support

Provides robust **governance** through comprehensive audit trails, role-based access control (RBAC), and enforceable policies.

Core Concepts



Registered Model

A named artifact family (e.g., fraud-detector) that groups related model versions

Version

Immutable snapshot containing model weights and associated metadata

Stage

None, **Staging**, **Production**, or **Archived** status

Metadata

Parameters, metrics, data references, git SHA, environment details, evaluation reports

What a Good Registry Stores

Artifacts

- Model files
(TorchScript/ONNX/TF)
- Inference code
- Containerized dependencies

Metrics

- Offline evaluation scores
- Fairness/robustness measures
- Calibration statistics

Lineage

- Dataset versions
- Feature views
- Code commit references
- Pipeline run identifiers

Constraints

- Schema definitions
- Input/output specifications
- Pydantic/OpenAPI contracts

Risk/Safety

- Bias test results
- PII handling documentation
- Red-team evaluation notes

A comprehensive registry captures not just the model itself but its entire context for safe deployment and auditability.

Popular Implementations



MLflow Model Registry

Open-source platform with managed options, widely adopted for its simplicity and flexibility



SageMaker Model Registry

AWS-native solution with tight integration to other AWS ML services



Vertex AI Model Registry

Google Cloud's offering with strong integration to BigQuery and GCP ecosystem



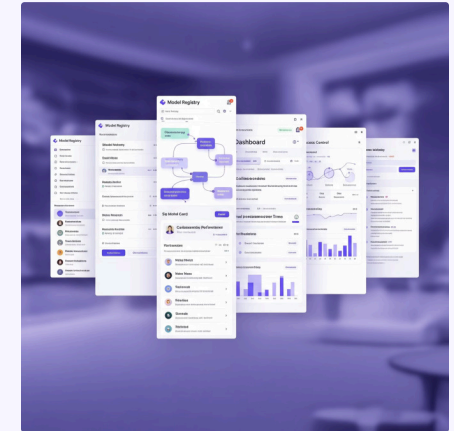
Azure ML Registry

Microsoft's enterprise-focused registry with governance capabilities



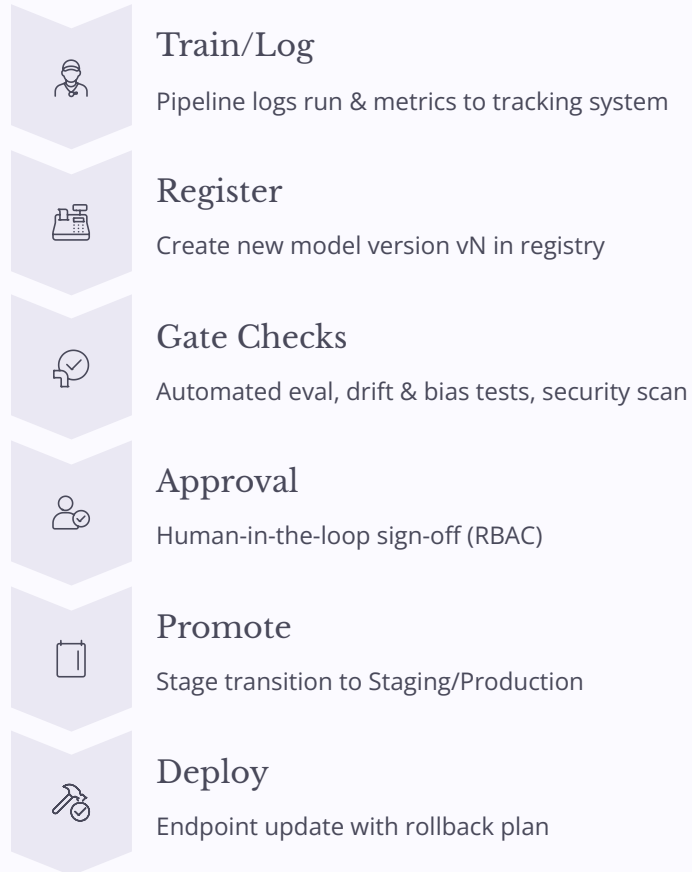
Databricks Unity Catalog

Governed registry across multiple asset types (models, data, notebooks)



Different registries offer varying levels of integration with their respective cloud ecosystems

Promotion Workflow (Typical)



Policy-as-Code (Gates)

Automated guardrails ensure only quality models reach production environments:

Performance Thresholds

Require **minimum AUC/F1** scores by demographic segment

Data Freshness

Enforce **training data recency** $\leq N$ days old

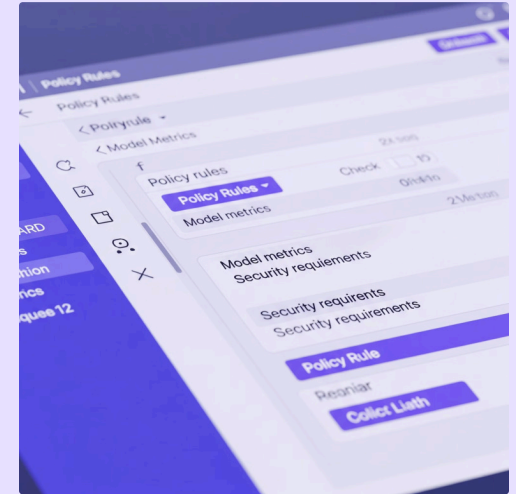
Drift Detection

Verify **feature schema** & **data drift bounds** are within tolerance

Security Compliance

Run **security scans** (licenses, CVEs) on artifacts/containers

 **Key Principle:** Block promotion if any gate fails; attach evidence to the version for audit trail



Policy gates coded as executable rules that must pass before promotion

Example (Pseudocode / MLflow-like)

```
# after training & evaluation
mv = mlflow.register_model(
    model_uri="runs://model",
    name="fraud-detector"
)

# attach metadata
mlflow.client.update_model_version(
    name="fraud-detector",
    version=mv.version,
    description=f"git={GIT_SHA}, data={DATASET_TAG}, auc={AUC:.3f}"
)

# policy gate
assert AUC >= 0.92 and max(abs(drift)) < 0.05

# promote
mlflow.client.transition_model_version_stage(
    name="fraud-detector",
    version=mv.version,
    stage="Staging"
)
```

This example demonstrates a typical workflow using MLflow's API to:

1. Register a newly trained model
2. Attach critical metadata for lineage
3. Enforce policy gates with code checks
4. Promote the model to staging after validation

The same principles apply across different registry implementations, though API details vary.



CI/CD Integration



Train Job

Logs training run and creates candidate model version in registry



CI Pipeline

Executes gates including unit tests, data validation, evaluation metrics, and security scans



CD Pipeline

On approval, transitions model stage and triggers deployment automation



Canary Deployment

Routes 5-10% traffic to new version; auto-rollback on p95/p99 latency or KPI regressions

Observability & Feedback

Serving Telemetry

Link registry version to production metrics like latency, error rates, and throughput

Live Metrics

Log business KPIs such as conversion rates, loss ratios, and false positives by demographic cohort

Shadow Testing

Automatically create shadow deployments for A/B comparison before full promotion

Feedback Loop

Feed real-world drift and performance signals back to retraining pipelines



Robust observability connects model versions to business outcomes, enabling data-driven promotion decisions.

Governance & Compliance

RBAC

Define who can register, promote, and archive models with granular permissions

Version Freeze

Support for regulated releases with reproducible rebuilds and immutability



Audit Trail

Track who approved changes, when, and with what supporting evidence

Retention Policies

Implement PII handling and right-to-be-forgotten hooks for compliance

A robust registry supports regulatory requirements across healthcare, finance, and other regulated industries.

Multi-Env & Multi-Cloud



Registry Architecture Options

- **Environment Strategy:** Choose between separate registries per environment or [single global registry](#) with logical partitions
- **Cross-Region Replication:** Mirror artifacts across regions to address latency and data sovereignty requirements
- **Portability:** Use portable formats (ONNX, containers) to reduce vendor lock-in
- **Deployment Mapping:** Connect registry events to provider-specific deployment systems (SageMaker/KServe/Vertex)

❏ Enterprise MLOps often requires spanning dev/test/prod environments across multiple cloud providers or regions.

Common Pitfalls

“

File Storage Mindset

Treating registry as a simple **file store** without leveraging metadata and gates, missing the core governance value

”

“

Schema Negligence

Missing **schema contracts** between model versions leads to production breakages when input/output expectations change

”

“

Rollback Blindness

Failing to implement and test rollback strategies for failed canary deployments, leaving production vulnerable

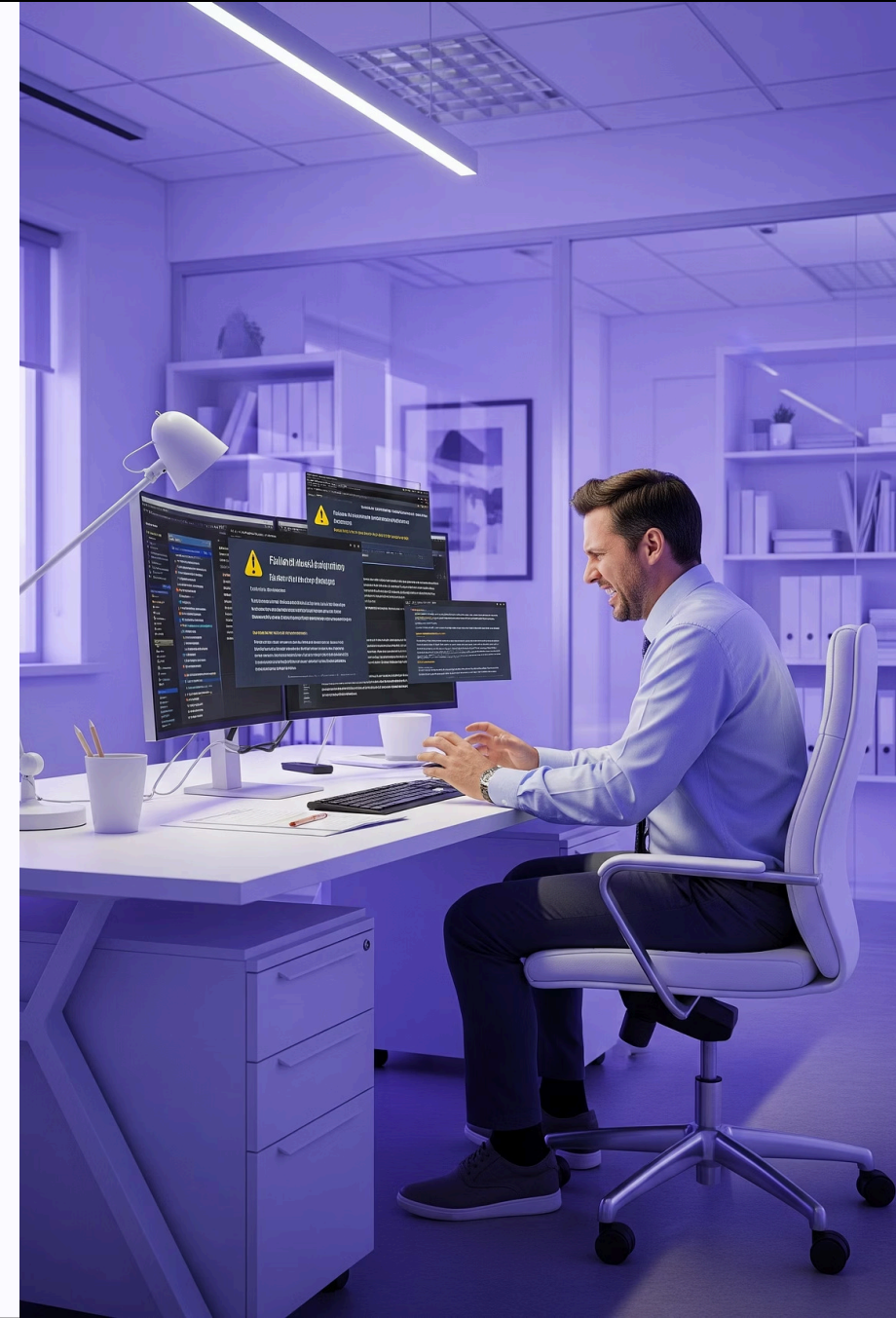
”

“

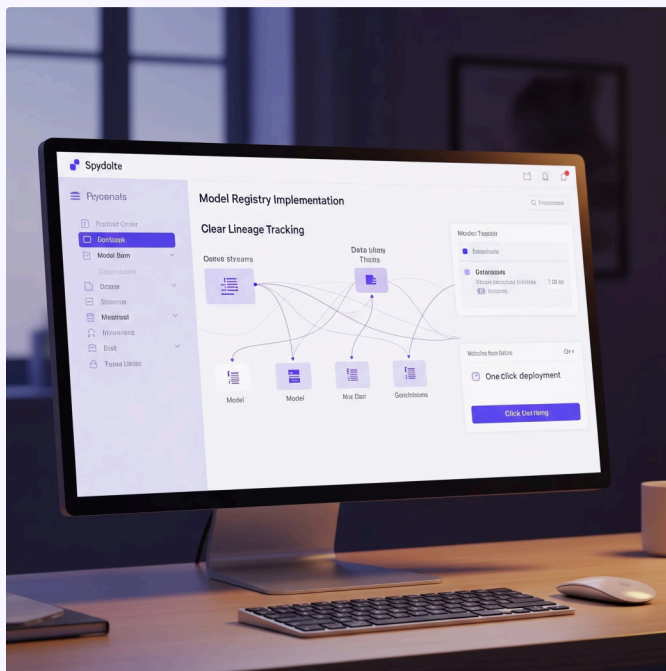
Orphaned Versions

Unclear ownership and lifecycle management creates "zombie" models with no clear RACI matrix or SLAs

”



What "Good" Looks Like



1 Registry-Driven Endpoints

Every production endpoint points to a [specific registry version](#) with clear provenance

2 Gated Promotion

Promotion requires both [automated gates](#) and [human approval](#) with clear accountability

3 Complete Lineage

Clear traceability from [raw data](#) → [features](#) → [model](#) → [endpoint](#) for every deployment

4 Safety Mechanisms

One-click [rollback](#) to previous Production version with no downtime

Key Takeaways

Model Registry: The Foundation of Enterprise MLOps

Source of Truth

Registry serves as the definitive repository for models and promotion decisions

Governance Backbone

Combine [metadata](#) + [policy-as-code](#) + [RBAC](#) to enable safe, compliant model releases

DevOps Integration

Wire into [CI/CD pipelines](#) and [monitoring systems](#) for continuous reliability

Strategic Investment

Essential infrastructure for scalable, governed Enterprise MLOps that reduces risk while accelerating innovation

❏ The registry is not just a technical tool but an organizational capability that enables ML governance at scale.